```
// routes/notes.js
// Handles adding notes to a case,
and searching across notes.

const express = require('express');
const pool = require('../db/pool');

const router = express.Router();

// Helper: confirms a case belongs to
the logged-in user before we touch
its notes.
// This prevents one user from
adding/reading notes on someone
else's case.
async function userOwnsCase(caseId,
userId) {
  const result = await pool.query(
    'SELECT id FROM cases WHERE id =
$1 AND user_id = $2',
    [caseId, userId]
  );
  return result.rows.length > 0;
}

// GET /api/notes/case/:caseId
```

```
// Returns all notes for a specific
case.
router.get('/case/:caseId', async
(req, res) => {
  const { caseId } = req.params;

  const owns = await
userOwnsCase(caseId, req.userId);
  if (!owns) return
res.status(404).json({ error: 'Case
not found.' });

  try {
    const result = await pool.query(
      'SELECT * FROM notes WHERE
case_id = $1 ORDER BY created_at
DESC',
      [caseId]
    );
    res.json(result.rows);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error:
'Something went wrong.' });
  }
});

// POST /api/notes
// Adds a new note to a case.
```

```
router.post('/', async (req, res) =>
{
  const { case_id, content, tags } =
req.body;

  if (!case_id || !content) {
    return res.status(400).json({
error: 'case_id and content are
required.' });
  }

  const owns = await
userOwnsCase(case_id, req.userId);
  if (!owns) return
res.status(404).json({ error: 'Case
not found.' });

  try {
    const result = await pool.query(
      'INSERT INTO notes (case_id,
content, tags) VALUES ($1, $2, $3)
RETURNING *',
      [case_id, content, tags ||
null]
    );

res.status(201).json(result.rows[0]);
  } catch (err) {
    console.error(err);
```

```
    res.status(500).json({ error:
'Something went wrong.' });
  }
});

// GET /api/notes/search?q=keyword
// Searches note content across all
of the user's cases.
router.get('/search', async (req,
res) => {
  const { q } = req.query;

  if (!q) return
res.status(400).json({ error: 'Query
param "q" is required.' });

  try {
    // to_tsquery / to_tsvector power
full-text search - much better than a
plain
    // LIKE '%keyword%' search, since
it understands word forms (e.g.
"filed" matches "filing").
    const result = await pool.query(
      `SELECT notes.*,
cases.client_name
       FROM notes
       JOIN cases ON notes.case_id =
cases.id
```

```
       WHERE cases.user_id = $1
         AND to_tsvector('english',
notes.content) @@
plainto_tsquery('english', $2)
       ORDER BY notes.created_at
DESC`,
      [req.userId, q]
    );
    res.json(result.rows);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error:
'Something went wrong.' });
  }
});

module.exports = router;
```